Genetic Algorithms for Test Suite Generation

EvoSuite

Announcements

- HW5 released
  - Due Monday November 2nd

- Exam Graded

Overview

- Midterm

- Genetic Algorithms
  - Employee scheduling problem

- EvoSuite - an **E**volutionary algorithm to generate test **S**uites

Midterm

Mean 61.1
Median 57
Std dev 15.7

Will be curved

Foundations

Undecidability of Program Analysis

In summary, it's impossible!

Sound + Complete + Terminating

Soundness - if a bug is present, the analysis will report it.
Completeness - if the analysis reports a bug it is truly a bug.

- AOB - inputs 20 and 21
- JUnit tests have @test annotation
- Should call readChannel(20) to trigger
  - No assertion needed. It wouldn't be executed anyway!
- 100% statement coverage is just readChannel with a non-triggering input
  - readChannel(10)
  - (anything 0-19 inclusive)
  - Assertion makes sense here but not necessary

Java Bytecode

Common mistakes:

- For loop / while loop
- Order of operations (2*x vs x*2)

AST

Common mistakes:

- Root should be if-stmt!
- IfStmt children:
  - Condition and body
- AssignmentStmt is its own node!
  - Children are LHS and RHS

Bytecode Transformations

Common mistakes:

- End of try block typically has a goto so we avoid entering the catch if an exception does not occur
- This example **does not have a goto** at the end of the try block. Why?
- Many of you forgot the astore
  - astore also needs a LVT slot! (astore_2)
- Target in the exception table should be after the return and before the astore

Coverage

- Branch coverage: each if statement has 2 branches
  - True case and false case

- Coverage vs Mutation score:
  - Coverage measures quality of the prefix!
  - Mutation score measures quality of the assertion!

- A test suite can have 100% branch coverage without any assertions
  - This is not great for bug-finding.
  - Only finds bugs that manifest as exceptions

Test Oracles

- Regression testing:
  - The regression oracle a tool adds should use the generated prefix!
  - If the generated assertion captures a bug that is later fixed, the test **will fail** on the fixed version.
- IR-based Retrieval
  - An incorrect oracle is retrieved since Jaccard weights variable names equally to operations
- Grammar of Oracles:
  - Derivation:
    - Start with the starting symbol (<oracle>)
    - each non-terminal gets replaced in each line of the derivation
  - Why use a grammar?
    - Best answer: to make the search space smaller
    - But I accepted many things... compilation, correctness, evaluation, etc.

Mutation Testing

Common mistakes:

- List which test killed each mutant
- Equivalent mutations

Randoop

Common Mistakes:

- constructor must be called before calling push / pop / other class methods
- Redundant sequences do not get added to extensibleSeqs
- 100% branch coverage is not always possible!
  - Infeasible branches
  - Unbounded / infinite branches

Search Based Software Testing

Idea: A test exists in a finite space of all possible Java tests. We just need to find the one which triggers the bug.

This requires two things

1) Prefix
2) Assertion

Genetic Algorithms

- A search and optimization technique inspired by natural selection and genetics
- Effective for complex search spaces
- NO CONVERGENCE GUARANTEES
- Termination: set number of iterations, time limit, or reached a “good enough” solution

Basic Genetic Algorithm

- Start with a large "population" of randomly generated "attempted solutions" to a problem
- Repeatedly do the following:
  - Evaluate each of the attempted solutions
  - (probabilistically) keep a subset of the best solutions
  - Use these solutions to generate a new population
- Quit when you have a satisfactory solution (or you run out of time)

Example: Employee Scheduling Problem

Imagine you're managing a small team of **3 employees**, and you need to create an optimal weekly schedule.

Each employee can either work a shift (1) or have the day off (0).

The objective is to create a schedule that balances **coverage** (you want as many people as possible working each day) and **minimizing overtime** (no one employee should work more than 4 days)

Genetic Algorithms

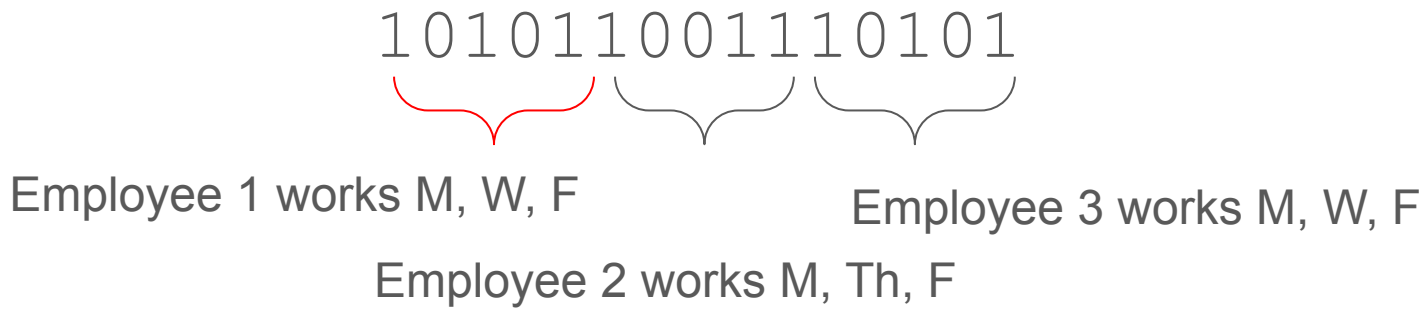
To adapt GA to any task we need to define the following:

1. What is a chromosome / individual in the population?
    a. This is a potential solution
2. How are individuals selected for breeding?
3. What happens during crossover?
4. What happens during mutation?
5. How is fitness evaluated?
    a. How do we measure how “good” a chromosome / individual is?
    b. Called the *fitness function*

Example: Employee Scheduling Problem

Encoding: a binary string of length 15

Each employee's schedule is represented by a binary string of length 5



Employee Schedule Fitness

Function that maps an individual (a potential solution) to a scalar value (fitness score). The higher the score, the better the solution.

1. **Maximize coverage**: For each day, if at least 2 employees are scheduled, the day contributes positively to the fitness.
2. **Minimize overtime**: For each employee, penalize if they work more than 4 days.

f(i) = # of days with at least 2 employees on each day - # of employees that work more than 4 days

Selection

How are individuals from the population selected for reproduction?

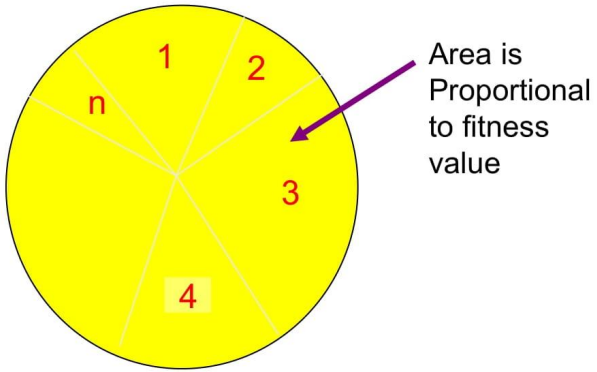
Step 1: Selection

We randomly (using a biased coin) select a subset of the individuals based on their fitness:

Individual *i* will have a probability to be chosen $\frac{f(i)}{\sum_i f(i)}$

S1 = 110101011011011
S2 = 101100101011011
S3 = 0111100111100101
S4 = 1110001110101000
S5 = 0000010100110001

...

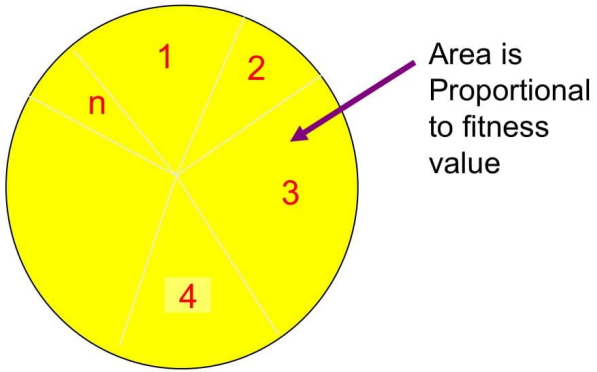


Step 1: Selection

We randomly (using a biased coin) select a subset of the individuals based on their fitness:

Individual *i* will have a probability to be chosen $\frac{f(i)}{\sum_i f(i)}$

S1 = 1101010010111111
f(1) = mon + tues + thurs - e3 = 2
S2 = 1001011000000001
f(2) = mon = 1
S3 = 0011101011111100
f(3) = tues + wed + thurs + fri = 4
S4 = 1110001010100000
f(4) = mon + tues = 2
...



This is called **roulette wheel selection**

Step 2: Crossover

Combines two parent solutions (individuals) to produce offspring solutions. The idea is to combine the good features of both parents to create one or more new solutions that may be better than the parents

Usually performed with some probability

Crossover for Employee Scheduling

Schedule1 x Schedule2 => creates two children

1. First 10 digits from Schedule1 and last 5 digits from Schedule2
    a. Take schedule for employees A and B from schedule 1 and C from schedule
2. First 10 digits from Schedue2 and last 5 digits from Schedule1

Step 3: Mutations

Mutation introduces small random changes to candidate solutions to maintain diversity

Performed on the offspring

Performed with some probability

Mutations for Employee Scheduling

1. Randomly select an index
2. Flip bit for that index

Basic Genetic Algorithm

- Start with a large "population" of randomly generated "attempted solutions" to a problem
- Repeatedly do the following:
  - Evaluate each of the attempted solutions
  - (probabilistically) keep a subset of the best solutions
  - Use these solutions to generate a new population
- Quit when you have a satisfactory solution (or you run out of time)

GA Handout

Use parameters we just went over.

Other parameters of our algorithm:

- Initial population size of 10
- Offspring are *added* to the population
- Parents are never removed
- Population only grows
- Termination occurs after 3 iterations

Step 0: Initialization

Generate a large random “population”

S0 = 110101011001010	S5 = 1100110100011111
S1 = 101100101011011	S6 = 011101001010110
S2 = 0101100111100101	S7 = 1001011110000011
S3 = 1110001110101000	S8 = 0010111001101101
S4 = 0011010101110001	S9 =1110100101011101

Selection script

Fitness Proportionate Selection: Each individual's chance of being selected is proportional to its fitness. Higher fitness means a higher likelihood of being chosen.

Normalized Fitness: First, we compute normalized fitness by dividing each individual's fitness by the total fitness. This gives us relative probabilities for each individual. For example, if an individual has a fitness of 10 and the total fitness is 55, the probability of selecting that individual is 10/55

Cumulative Probabilities: Let's say we have 4 individuals with normalized fitness values: [0.2,0.3,0.4,0.1]

Their **cumulative probabilities** would be calculated as:
[0.2, 0.2+0.3=0.5 ,0.5+0.4=0.9,0.9+0.1=1.0]

So, the cumulative probabilities are:
[0.2,0.5,0.9,1.0]
This means:

- Individual 1 occupies the range [0.0,0.2]
- Individual 2 occupies the range (0.2,0.5]
- Individual 3 occupies the range (0.5,0.9]
- Individual 4 occupies the range (0.9,1.0]

Genetic Algorithms in Test Suite Generation

EvoSuite

Genetic Algorithms for Test Suite Generation

To adapt GA to any task we need to define the following:

1. What is a chromosome / individual in the population?
   a. A test case!
2. How is fitness evaluated?
   a. Coverage!
3. How are individuals selected for breeding?
   a. “Rank Selection”
4. What happens during crossover?
   a. Take part of test1 and part of test2
5. What happens during mutation?
   a. Remove a statement, swap out a call, replace a primitive, etc....

Initial Population

How do we generate a random initial population of test cases?

- Select a random length n
- Randomly insert n statements
- Many are invalid!
- Many are redundant

We need to do this in a more principled way... Java has rules

How did Randoop do this?

Initial Population

When inserting random statement either:

1. Insert a call to the unit under test
   a. constructor or method
   b. Parameters: re-use existing values in the test or create a new variable

2. Insert a statement which can later be used as a parameter to a call to the unit under test

Fitness

Fitness = Number of branches covered / total branches

Are some of these conflicting?

We will discuss how to properly balance multiple objectives in GA next class

Rank Selection

A method for choosing individuals to be parents for the next generation where

1. the population is first ranked based on their fitness, and then
2. selection probabilities are assigned based on their rank

This means the best individuals (with the highest rank) have a higher chance of being selected, while the worst individuals have a lower chance, regardless of the exact difference in their fitness values

How is this different from roulette? Pros / Cons?

Rank Selection

1. Calculate fitness
2. Rank the population
    a. Sort the individuals based on their fitness values, assigning a rank to each individual (e.g., 1 for the best, N for the worst, where N is the population size)
3. Assign selection probabilities
    a. Based on the assigned ranks, determine the selection probability for each individual.
    - **Linear ranking:** A simple approach where the selection probability is directly proportional to the rank (best individual has the highest probability).

4. Select Parents
    a. Randomly select individuals for reproduction based on their assigned selection probabilities.

Evosuite avoids scaling and cumulative probability by approximating the rank

Crossover

How do we create an offspring of two test cases?

1. Generate a random number between 0 and 1. This is the % of statements from each test to keep. If test1 has n1 statements and test2 has n2 statements
   a. `k = floor(r * n1) , j = floor(r * n2)`

2. New test1 = the first k statements from test1 and the last k statements from test2.
3. New test2 = the first j statements from test2 and the last j statements from test1.

4. Clean up test by removing unused variables

Crossover

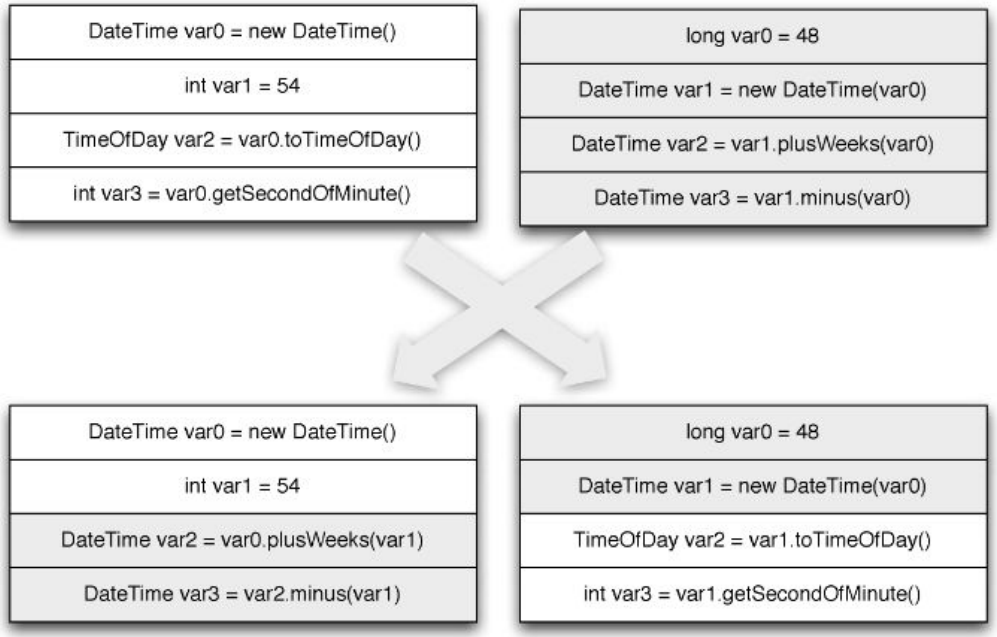


Fig. 4. Crossover between two test cases.

Mutation

1. Delete a statement
2. Insert a method call
3. Modify an existing statement
    a. Change callee (target object)
    b. Change params
    c. Change method / constructor - replace with one of the same return type
    d. Change field
    e. Change primitive

How do you think these program transformations are implemented?

Assertions

Test suites contain a prefix and assertion

The GA we defined only generates a prefix. How does it find the assertion?

EvoSuite generates ***regression*** tests.

Multiple Objective Optimization

Genetic Algorithms with Multiple Objectives

Often times, our fitness function includes multiple objectives which may be conflicting.

- Meal Plan: Maximize nutritional value, minimize cost
- Employee Scheduling: Maximize coverage, minimize overtime
- Test Suite Generation: Maximize branch coverage

```
String example(int a) {
  switch (a) {
     case  0 : return "0";
     case  1 : return "1";
     case -1 : return "-1";
  default: return "default";
}
```

Genetic Algorithms with Multiple Objectives

Fitness functions for multiple objectives:

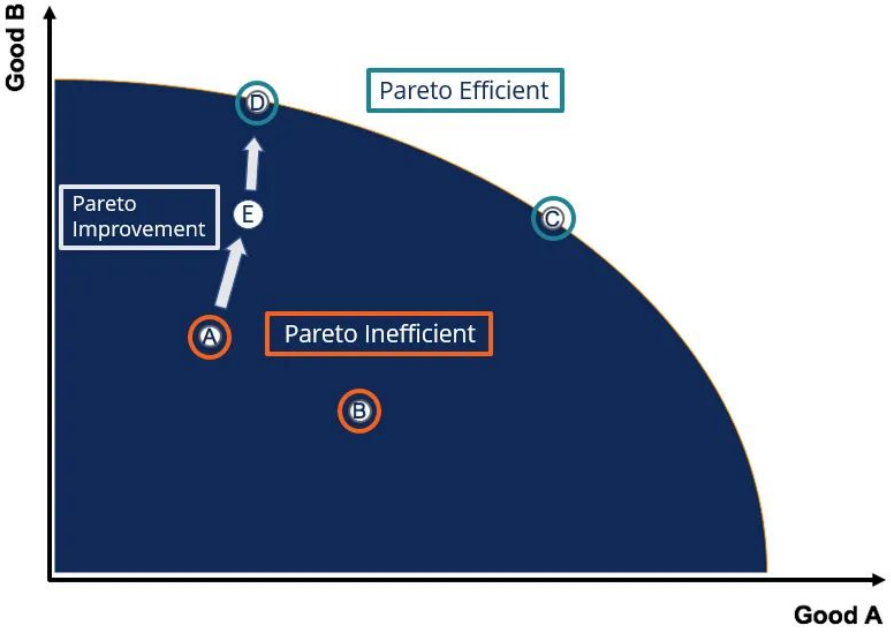
1. Weighted linear combination
    a. Nutritional Value + .5 * cost
    b. This becomes less effective with many goals
    c. Programs often have 100s of branches...

Pareto Fronts

A **Pareto improvement** formalizes the idea of an outcome being "better in every possible way".

A change is called a **Pareto improvement** if it leaves at least one person in society better-off without leaving anyone else worse off than they were before.

A situation is called **Pareto efficient** or **Pareto optimal** if all possible Pareto improvements have already been made; in other words, there are no longer any ways left to make one person better-off, without making some other person worse-off



Pareto Dominance

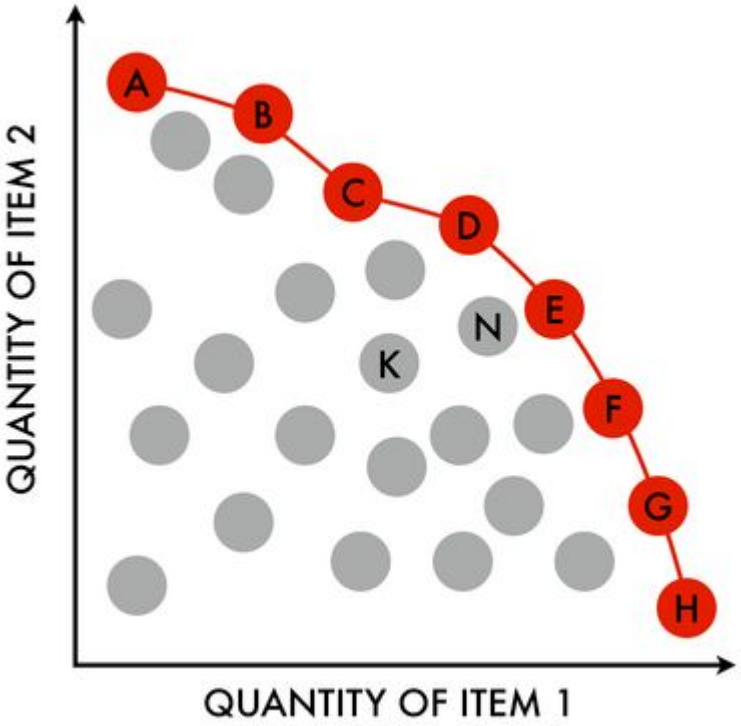
Pareto Dominance: A solution A dominates solution B if A is at least as good as B in all objectives and strictly better in at least one.

Pareto Front: The set of non-dominated solutions representing the trade-offs between conflicting objectives.

Production Possibility Frontier

The red line is an example of a Pareto-efficient frontier, where the frontier and the area left and below it are a continuous set of choices.

The red points on the frontier are examples of Pareto-optimal choices of production. Points off the frontier, such as N and K, are not Pareto-efficient, **since there exist points on the frontier which Pareto-dominate them.**

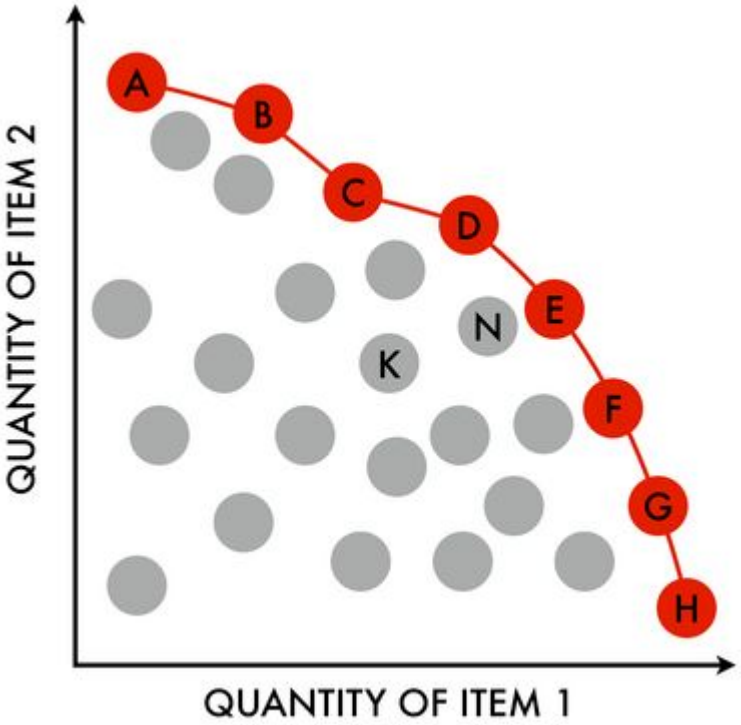


Production Possibility Frontier

Points off the frontier, such as N and K, are not Pareto-efficient, **since there exist points on the frontier which Pareto-dominate them.**

a solution A dominates solution B if A is at least as good as B in all objectives and strictly better in at least one.

What points dominate K and N?



Back to EvoSuite

EvoSuite uses these concepts in its **selection function**

Calculates pareto optimal solutions

a. Tests which are not “dominated”
b. Dominance is when one test has better fitness on some branch and the other is not better on any other branch
c. If T1 coverages branch 4F, but T2 covers branch 4T, and are the same on all other branches, neither domains the other

Branch Coverage Dominance Example

```
public void unlock(String pin) {
        //check if account holder is admin
        if (pin.equals(decode(admin))) { //B1
                locked = false;
                return;
        }
}
public boolean withdraw(double amount) {
        if (locked) return false; //B2

        if (amount > 0 && balance >= amount) { //B3
                balance -= amount;
                return true;
        }
        return false;
}
```

Dominance is when one test has better fitness on some branch and the other is not better on any other branch

Test Case	Branches Covered
T1	B2 T
T2	B1 F, B2 T
T3	B1 T, B2 F, B3 F

T2 dominates T1

T3 and T2 do not dominate eachother

Making progress on B3 would make progress on B2 F worse

EvoSuite Selection and Fitness

Last class, we simplified fitness: `total branches covered / total branches`

It's actually sorted by dominance ! All tests on the pareto front are equal

Higher branch coverage doesn't mean the test is necessarily better! We want our test suite to include both cases:

- Call to withdraw with locked
- Call to withdraw with unlocked

DynaMosa

We want both T2 and T3 in our test suite, but if we keep B1F as a goal, we will never make progress toward covering B3T

DynaMosa dynamically adapts which goals are included in our ranking

Add T2 to the test suite and remove B1F from our goals

DynaMosa

1. Generate initial population

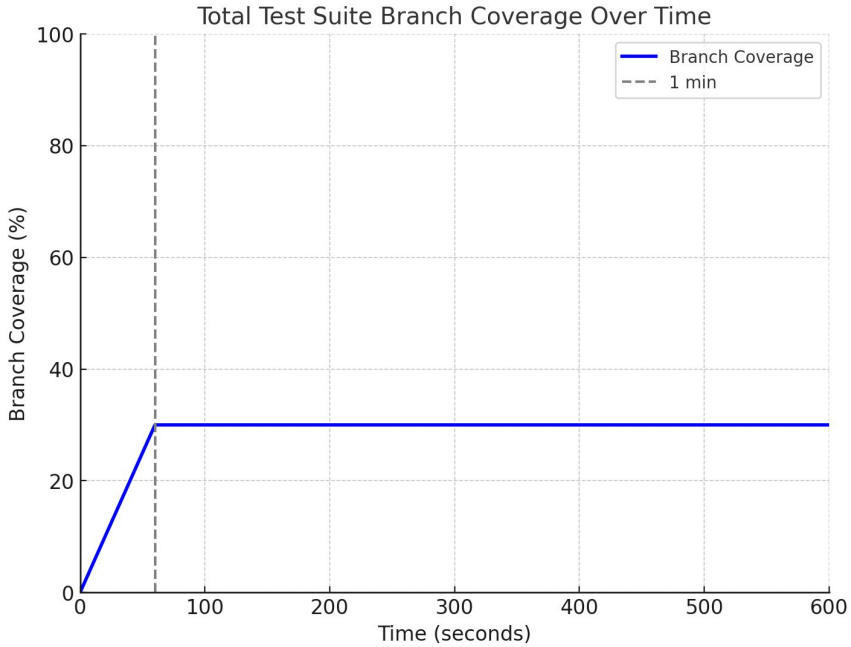
2. Until timeout:
   a. Select Parents
      i. **Requires evaluating population and ranking**
      ii. Rank based selection
   b. Perform Crossover
   c. Perform mutation with some probability
   d. Dynamically adapt goals

3. Output test suite

CodaMosa: Escaping Coverage Plateaus with LLMs

Coverage Plateaus

- SBST works well when mutation / crossover has a non-negligible likelihood of increasing fitness
- Imagine our unlock method...
  - What are the odds we would call unlock with the “magic pin” ?
- Fitness plateaus are common problem in test generation



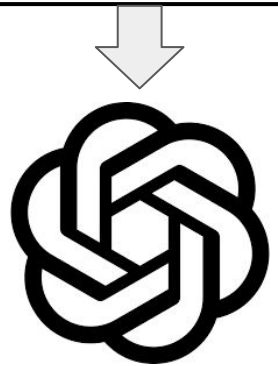
CodaMosa

CodaMOSA, starts SBST and monitors its coverage progress.

When it notices a stall in coverage, it identifies methods that have low coverage

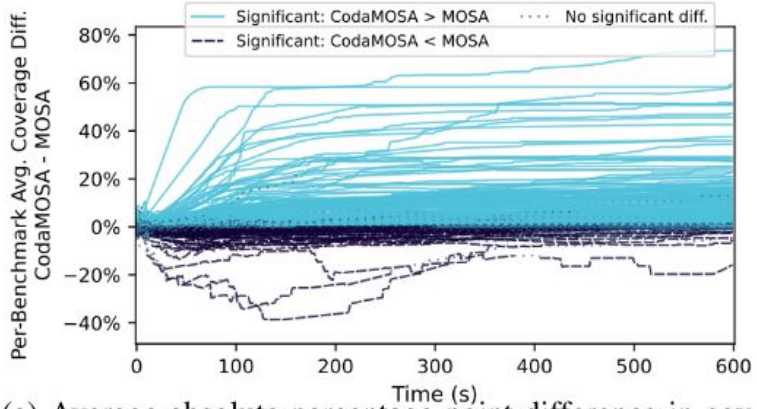
Then, it queries Codex to generate tests for these methods and adds them to the population

```
public void unlock(String pin) {
        //check if account holder is admin
        if (pin.equals(decode(admin))) { //B1
                locked = false;
                return;
        }
}
```

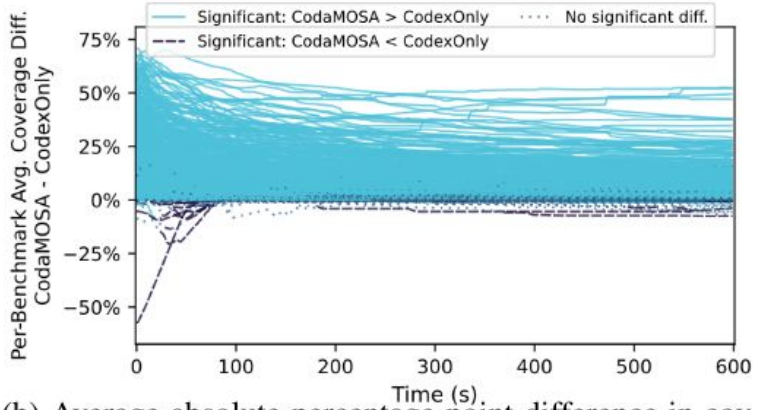


unlock(“admin”)

CodaMosa Evaluation



(a) Average absolute percentage point difference in coverage between CODAMOSA and MOSA.



(b) Average absolute percentage point difference in coverage between CODAMOSA and CodexOnly.

Summary

- Randoop generates low quality tests (both prefix and assertion)
- EvoSuite guides search based on coverage fitness value
- We traced a GA on employee scheduling
- We adapted GA to test suite generation by defining various aspects:
  - Population, selection, crossover, mutation ...